

IZZIMA: AI 기반 개인 갤러리 관리 시스템



팀명: 19팀 줄크크

B 분과: 데이터베이스/그래픽스 및 비전

202355621 문진서 jshin27@pusan.ac.kr

202355555 안지원 wldnjs7805@pusan.ac.kr

202355652 이태경 taekoong@pusan.ac.kr

지도교수: 조준수

제출일: 2026년 9월

부산대학교

목차

1	서론	1
1.1	연구 배경 및 기존 문제점	1
1.2	연구 목표	1
2	연구 배경	2
2.1	이미지 기반 정보 추출 기술	2
2.2	대규모 언어 모델 기반 정보 추출	2
2.3	임베딩 기반 의미 검색	3
3	연구 내용	4
3.1	전체 시스템 구조	4
3.2	AI 기반 이미지 분석 시스템	5
3.3	자연어 이미지 검색 시스템	8
3.4	백엔드 시스템 구현	10
3.5	프론트엔드 시스템 구현	13
3.6	시스템 통합 및 서비스 구현	19
4	연구 결과 분석 및 평가	21
4.1	평가 환경	21
4.2	AI Pipeline 결과 분석	22
4.3	Search Text 구성 방식 비교	23
4.4	시스템 구현 결과	25
4.5	개발 일정 및 역할 분담	25
4.6	오류 사례 및 한계 분석	26
5	결론 및 향후 연구 방향	28
5.1	연구 결과 요약	28
5.2	향후 연구 방향	28
6	참고 문헌	30

제1장 서론

1.1 연구 배경 및 기존 문제점

스마트폰 사용이 증가함에 따라 사용자는 모바일 쿠폰, 예약 확인서, 승차권, 시험 일정, 과제 공지 등 일상생활에 필요한 정보를 이미지와 스크린샷 형태로 저장하고 있다. 이러한 이미지에는 날짜, 장소, 금액, 좌석, 만료일 등 실제 행동과 연결되는 중요한 정보가 포함되어 있다. 그러나 일반적인 갤러리 애플리케이션은 촬영 시점이나 폴더 분류를 중심으로 이미지를 관리하기 때문에 이미지 내부의 내용을 기억하지 못하면 필요한 자료를 찾기 어렵고, 쿠폰 만료일이나 예약 일정과 같이 시간에 민감한 정보도 사용자가 직접 확인하지 않는 경우 놓칠 수 있다. OCR은 이미지의 문자열을 추출할 수 있지만 날짜, 장소, 상품명 등의 의미를 판단하고 구조화하기 어렵고, 이미지 캡셔닝은 시각적 문맥을 설명할 수 있으나 날짜, 예약번호, 금액과 같은 세부 문자 정보 추출에는 한계가 있다. 따라서 두 정보를 함께 분석하여 검색과 일정 관리에 활용할 수 있는 구조화된 정보로 변환하는 시스템이 필요하다.

1.2 연구 목표

본 프로젝트에서는 스마트폰 갤러리의 이미지를 검색과 일정 관리에 활용할 수 있는 정보로 변환하는 AI 기반 개인 갤러리 관리 서비스 **IZZIMA**를 구현하였다. OCR과 이미지 캡셔닝을 통해 문자 정보와 시각적 문맥을 추출하고, 대규모 언어 모델을 이용하여 이를 공통 Event Schema로 구조화한다. 또한 모델 출력에 대한 검증과 후처리를 통해 누락되거나 불완전한 정보를 보완한다. 구조화된 Event는 Search Text로 변환한 뒤 임베딩 기반 의미 검색에 활용하여 사용자가 이미지 속 정확한 문구를 기억하지 못하더라도 자연어로 필요한 이미지를 찾을 수 있도록 하며, Event를 일정 관리와 연계하여 쿠폰 만료, 예약, 시험 및 과제와 같은 시간에 민감한 정보를 관리할 수 있도록 한다.

제2장 연구 배경

2.1 이미지 기반 정보 추출 기술

2.1.1 광학 문자 인식

광학 문자 인식(Optical Character Recognition, OCR)은 이미지에 포함된 문자 영역을 탐지하고 텍스트로 변환하는 기술이다. 본 프로젝트에서는 한국어 생활형 이미지에 포함된 날짜, 시간, 장소, 상품명, 주문번호 등의 문자열을 추출하기 위해 PaddleOCR을 사용하였다. OCR 결과는 이후 LLM이 이미지를 이해하기 위한 핵심 입력으로 사용되지만, OCR 자체는 이미지 내 텍스트의 의미 관계를 직접 해석하지 못하며 저해상도 이미지나 복잡한 배경에서는 일부 문자열이 잘못 인식되거나 누락될 수 있다.

2.1.2 이미지 캡셔닝

이미지 캡셔닝은 이미지의 시각적 내용을 자연어 문장으로 표현하는 기술이다. 본 프로젝트에서는 BLIP을 활용하여 이미지의 시각적 문맥을 캡션으로 생성하고 OCR Text만으로 파악하기 어려운 이미지의 전반적인 의미를 보조하도록 하였다. OCR이 문자 중심의 정보를 제공한다면 Image Caption은 이미지의 장면과 객체 정보를 제공하므로 두 정보를 함께 활용함으로써 이미지 분석의 입력 정보를 보완할 수 있다.

2.2 대규모 언어 모델 기반 정보 추출

대규모 언어 모델(Large Language Model, LLM)은 자연어 문맥을 이해하고 주어진 형식에 따라 정보를 생성할 수 있다. 본 프로젝트에서는 OCR Text와 Image Caption을 입력으로 제공하고 이미지가 일정 정보를 포함하는지를 판단한 뒤 미리 정의한 Event Type과 Event Schema에 맞추어 구조화된 정보를 생성하도록 하였다. 최종 구현에서는 Qwen 계열 모델을 사용하며, 일반적인 입력은 Qwen3.5 9B로 처리하고 OCR에 존재하는 중요 정보가 구조화 결과에서 누락되는 등 출력이 불완전하다고 판단되는 경우에만 Qwen3.5 27B를 추가적으로 호출하는 Primary-Fallback 구조를 적용하였다.

2.3 임베딩 기반 의미 검색

키워드 검색은 사용자의 검색어와 저장된 문자열 사이에 직접적인 문자열 일치가 필요한 경우가 많지만, 임베딩 기반 의미 검색은 자연어 문장을 벡터 공간에 표현하여 표현이 서로 다르더라도 의미적으로 가까운 텍스트를 검색할 수 있다. 본 프로젝트에서는 Event의 구조화 정보를 검색에 적합한 자연어 문자열인 Search Text로 변환한 후 한국어 임베딩 모델 KURE-v1을 이용하여 벡터화하였다. 사용자 질의 역시 동일한 임베딩 공간으로 변환하며, Query Embedding과 저장된 Image Embedding 사이의 코사인 유사도를 계산하여 관련성이 높은 이미지를 검색한다.

제3장 연구 내용

3.1 전체 시스템 구조

IZZIMA는 사용자가 스마트폰 갤러리의 이미지를 등록하면 AI가 이미지 내부의 텍스트와 시각적 정보를 분석하여 구조화된 Event를 생성하고 이를 자연어 검색과 일정 관리에 활용하는 서비스이다. 전체 시스템은 React Native 기반 모바일 인터페이스를 제공하는 Frontend, 사용자 인증과 API 처리 및 AI Pipeline 호출을 담당하는 Backend, OCR과 이미지 캡셔닝 및 LLM 기반 정보 추출을 수행하는 AI Pipeline, 이미지와 Event 및 검색용 벡터를 저장하는 Database로 구성된다. 사용자가 이미지를 등록하면 Backend가 AI Pipeline을 호출하여 분석 결과와 검색용 임베딩을 Database에 저장하며, 이후 자연어 질의를 동일한 임베딩 공간으로 변환하여 관련성이 높은 이미지를 검색한다.



그림 3.1: IZZIMA 전체 시스템 구조

3.2 AI 기반 이미지 분석 시스템

3.2.1 AI Pipeline 구성

본 프로젝트의 AI Pipeline은 이미지에서 문자 정보와 시각적 문맥을 추출하고 이를 구조화된 Event 정보로 변환한다. 먼저 PaddleOCR을 이용하여 날짜, 시간, 장소, 상품명, 좌석 및 주문번호와 같은 문자 정보를 추출하고, BLIP을 이용하여 이미지의 전반적인 시각적 문맥을 Image Caption으로 생성한다. OCR Text는 정확한 문자 정보의 주요 근거로 사용하고 Image Caption은 이미지의 종류와 문맥을 판단하는 보조 정보로 활용한다.

두 결과는 Qwen 기반 LLM에 전달되어 Event Type과 Event Schema에 따른 구조화 정보로 변환되며, LLM 출력은 스키마 검증과 규칙 기반 후처리를 거쳐 최종 Event로 사용된다. 이후 최종 Event를 기반으로 Search Text를 생성하고 KURE-v1을 이용하여 검색용 Embedding을 생성한다.

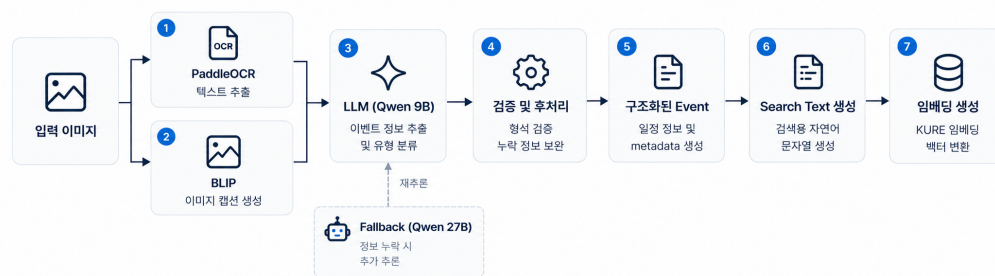


그림 3.2: AI 기반 이미지 분석 Pipeline

3.2.2 Event 분류 및 구조화 정보 추출

OCR Text와 Image Caption은 Qwen 기반 LLM의 입력으로 전달되며, LLM은 이미지가 일정 정보를 포함하는지를 판단하고 일정 정보가 존재하는 경우 적합한 Event Type을 선택하는 동시에 Event Schema에 따른 세부 정보를 추출한다. Event Type은 실제 스마트폰 갤러리에 저장되는 생활형 이미지와 일정 관리 기능을 고려하여 총 10개의 유형으로 정의하였다.

표 3.1: Event Type 정의

Event Type	설명
expiration	쿠폰, 상품권 등의 유효기간 및 만료 일정
exam	시험 일정
assignment_due	과제 제출 마감 일정
reservation	일반 예약 및 예매 일정
departure	항공권, 기차표 등 교통수단의 출발 일정
check_in	숙박 등의 체크인 일정
performance	공연, 콘서트, 영화 및 행사 일정
meeting	회의, 미팅 및 약속
schedule	위 유형에 포함되지 않는 일반 일정
none	일정 정보가 존재하지 않는 이미지

생활형 이미지에는 구매일, 사용일, 취소 기한, 유효기간과 같이 여러 날짜가 동시에 존재할 수 있으므로 단순히 날짜 문자열을 추출하는 것보다 Event Type과 이미지 문맥을 함께 고려하여 어떤 날짜가 실제 사용자가 관리해야 할 일정인지 판단할 필요가 있다. 이를 위해 LLM이 다음과 같은 Event Schema로 결과를 생성하도록 구성하였다.

표 3.2: Event 출력 스키마

Field	Description
type	이벤트 유형
title	이벤트 또는 이미지의 핵심 제목
date	이벤트가 발생하거나 유효한 날짜
time	이벤트 시작 시간
end_time	이벤트 종료 시간
location	이벤트 장소 또는 상품 사용처
metadata	이벤트 종류에 따른 추가 세부 정보

예약번호, 주문번호, 좌석, 금액과 같이 Event 종류에 따라 달라지는 세부 정보는 metadata 구조를 이용하여 저장하였다.

표 3.3: Event Metadata 주요 필드

Field	Description
reservation_number	예약 및 예매 번호
order_number	주문 번호
buyer_name	구매자 또는 예매자 이름
purchase_date	구매 또는 결제 날짜
raw_date	원본 이미지에 표시된 날짜 표현
cancellation_deadline	취소 가능 기한
screen	상영관 등의 세부 장소 정보
seats	좌석 정보
quantity	구매 또는 예약 수량
amount	상품권 및 쿠폰 등의 금액
purchase_amount	실제 구매 금액
ticket_amount	티켓 금액
status	예약 또는 주문 상태
receive_method	티켓 또는 상품 수령 방법
exchange_code	교환 및 사용 코드
format	티켓 또는 콘텐츠의 형식 정보

Event Schema는 AI가 이미지로부터 추출하는 일정 정보를 표현하기 위한 구조이며 애플리케이션의 이미지 폴더 분류에 사용하는 category와는 구분된다. AI Pipeline의 최종 Event Schema에는 별도의 category 필드를 두지 않으며, 사용자가 이미지를 분류하기 위한 category는 Backend의 images Table에서 독립적으로 관리한다.

3.2.3 Prompt 및 구조화 출력 설계

LLM이 자유로운 자연어 문장을 생성하도록 할 경우 출력 형식이 요청마다 달라져 Backend와 Database에서 안정적으로 처리하기 어렵기 때문에 Event Schema를 모델의 출력 구조로 지정하고 OCR Text와 Image Caption을 기반으로 정의된 필드만 반환하도록 하였다. Prompt에서는 OCR Text를 날짜, 시간, 장소, 상품명 등 정확한 정보의 주요 근거로 사용하고 Image Caption은 이미지의 종류와 전체적인 시각적 문맥을 판단하는 보조 정보로 활용하였다.

두 정보가 충돌하는 경우 OCR을 우선하며, Event Type에 적합한 날짜와 시간을 선택하고 예약번호, 좌석, 주문번호, 금액 등의 추가 정보는 metadata에 저장하도록 하였다. 이미지에서 확인할 수 없는 정보는 임의로 추측하지 않으며 일정 관련 정보가 존재하지 않는 경우에는 Event Type을 none으로 분류하도록 구성하였다. Search Text는 LLM이 직접 생성하지 않고 구조화된 Event 및 OCR, Caption 결과를 이용하는 별도의 모듈에서 생성하여 LLM 출력의 표현 차이를 줄이고 Search Text 구성만 독립적으로 변경하며 검색 성능을 평가할 수 있도록 하였다.

3.2.4 Primary-Fallback 모델 기반 추론

초기 구현에서는 하나의 LLM을 이용하여 모든 이미지의 Event 정보를 추출하였으나, OCR Text에 예약번호, 주문번호, 좌석과 같은 핵심 정보가 명확하게 존재함에도 일부 이미지에서 해당 정보가 구조화 결과에 포함되지 않는 사례를 확인하였다.

이를 해결하기 위해 일반적인 이미지 분석에는 Qwen3.5 9B를 Primary Model로 사용하고, OCR 원문에 중요 정보가 존재하지만 구조화 Event에서 해당 정보가 누락되는 등 결과가 불완전하다고 판단되는 경우에만 Qwen3.5 27B를 Fallback Model로 호출하도록 구성하였다. 이를 통해 대부분의 입력은 상대적으로 작은 모델로 처리하면서 필요한 경우에만 큰 모델을 사용하도록 하였다.

3.2.5 출력 검증 및 후처리

LLM이 전체 Event의 문맥은 올바르게 이해하더라도 OCR에 명시된 일부 정형 정보를 누락하는 사례가 확인되어 출력 검증과 규칙 기반 후처리를 적용하였다. 먼저 LLM 출력이 정의된 Event Schema와 일치하는지 검증하고, expiration Event에서 금액이 누락된 경우 OCR Text에서 “50,000원”, “3만원”, “5천원”과 같은 표현을 정규식으로 탐색하여 metadata의 amount를 보완하였다. 이를 통해 LLM의 문맥 이해 능력과 규칙 기반 정보 추출의 안정성을 함께 활용하여 최종 Event를 생성하였다.

3.3 자연어 이미지 검색 시스템

3.3.1 Search Text 생성

구조화된 Event Schema는 일정 정보를 저장하고 관리하기에는 적합하지만 JSON의 필드명이나 검색과 직접적인 관련성이 낮은 정보가 포함될 수 있어 이를 그대로 임베딩하는 것이 자연어 검색에 가장 효과적인 것은 아니다. 이에 따라 구조화 Event와 검색용 표현을 분리하고 Event의 핵심 필드 및 필요한 경우 OCR Text와 Image Caption을 조합하여 별도의 Search Text를 생성하였다. 어떤 정보를 Search Text에 포함하는지가 검색 성능에 영향을 미칠 수 있다고 판단하여 총 여섯 가지 구성 방식을 비교하였다.

3.3.2 Search Text 버전 설계

표 3.4: Search Text 버전별 구성

Version 구성	
V1	title + type + date + time + location
V2	V1 + end_time + metadata의 유효한 값
V3	V2 기반으로 raw Event Type 대신 Event Type별 자연어 검색 확장 표현 사용
V4	V3 + BLIP Image Caption
V5	일정 Event는 V3, none은 OCR Text + BLIP Image Caption 사용
V6	일정 Event는 V3, none은 OCR Text가 존재하면 OCR을 우선하고 없는 경우 Caption 사용

V1은 기본 Event 필드만을 사용하는 Baseline이며 V2에서는 종료 시간과 metadata를 추가하였다. V3에서는 내부 Event Type을 실제 사용자 질의와 가까운 자연어 표현으로 확장하였고, V4에서는 모든 이미지에 BLIP Image Caption을 추가하였다. 이후 Caption이 문서나 티켓 이미지에서 불필요한 노이즈로 작용할 수 있음을 고려하여 V5에서는 일정 Event에는 V3를 유지하고 none 유형에만 OCR Text와 Caption을 활용하였다. 최종 V6에서는 none 유형에서도 OCR Text가 존재하면 OCR을 우선하며 OCR이 없는 일반 사진에만 Caption을 사용하도록 구성하였다.

표 3.5: Event Type별 검색 확장 표현

Event Type	Search Expression
expiration	유효기간 만료 쿠폰 상품권
exam	시험 중간고사 기말고사
assignment_due	과제 마감 제출
reservation	예약 예매
departure	출발 교통
check_in	체크인 숙박
performance	공연 콘서트 행사
meeting	회의 미팅 약속
schedule	일정 스케줄
none	별도 확장 표현 없음

3.3.3 KURE 기반 임베딩 생성

생성된 Search Text는 한국어 임베딩 모델 KURE-v1을 이용하여 1024차원의 벡터로 변환한다. 본 서비스에서 다루는 OCR Text와 사용자의 검색 질의가 주로 한국어로 구성된다는 점을 고려하여 한국어 자연어 검색에 활용할 수 있는 KURE-v1을 사용하였다. 이미지를 처음 등록할 때 Search Text Embedding을 생성하여 저장하고, 사용자가 검색어를 입력하면 동일한 모델로 Query Embedding을 생성한다. 검색 단계에서는 Query Embedding과 저장된 Image Embedding 사이의 코사인 유사도를 계산하여 유사도가 높은 이미지부터 반환한다.

3.4 백엔드 시스템 구현

3.4.1 서버 구조

Backend는 Frontend와 AI Pipeline 사이에서 요청을 중계하고 사용자 인증, 이미지 및 Event 데이터 관리, 자연어 검색 및 Storage 연동 등의 기능을 담당한다. Python 기반 FastAPI를 사용하였으며, Pydantic을 통해 요청과 응답의 데이터 형식을 검증하고 OpenAPI 기반 API 문서를 개발 과정에서 활용하였다. 내부 구조는 HTTP 요청과 인증 및 예외 처리를 담당하는 `routes`, Database CRUD를 담당하는 `crud`, Pydantic 데이터 구조를 정의하는 `schemas`, AI Pipeline과 Embedding 및 Storage 연동을 담당하는 `services` 계층으로 구분하였다.

3.4.2 데이터베이스 설계

Database는 Supabase PostgreSQL을 기반으로 하며 자연어 검색을 위한 Vector 저장과 연산에는 pgvector 확장을 사용하였다. 주요 데이터는 `images`, `events`, `image_embeddings`의 세 Table로 구분하며, `images`가 사용자가 등록한 이미지에 대응하는 중심 Table이고 나머지 두 Table은 `image_id`를 통해 연결된다. 사용자 인증 정보는 Supabase Auth에서 관리하며 `images.user_id`를 통해 이미지 소유자를 구분한다.

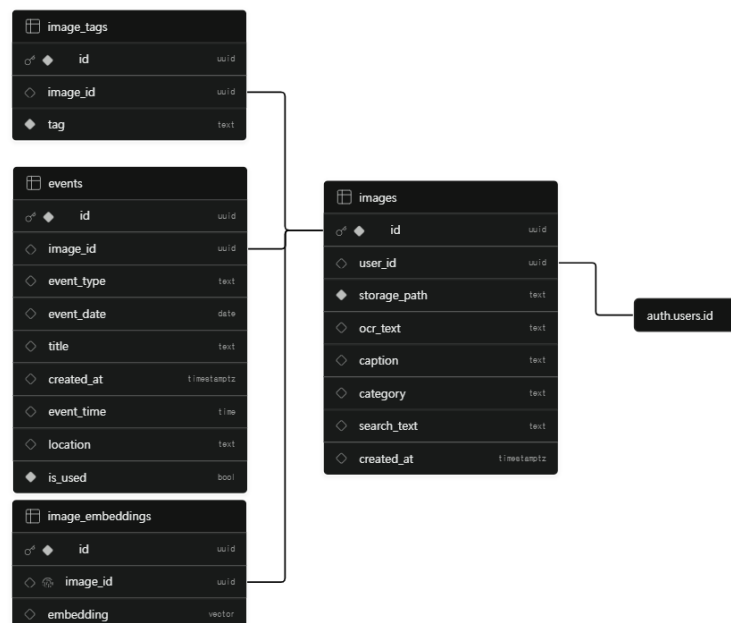


그림 3.3: 데이터베이스 ERD

images Table

images Table에는 이미지 원본의 Storage 경로와 AI Pipeline에서 생성한 OCR Text, Image Caption, Search Text 및 사용자 폴더 분류용 category를 저장한다. 여기서 category는 AI Event Type과 별개이며 사용자가 애플리케이션 안에서 이미지를 폴더 단위로 관리하기 위한 값이다.

표 3.6: images Table Schema

Column	Type	설명
id	uuid	이미지 고유 ID
user_id	uuid	이미지를 등록한 사용자 ID
storage_path	text	Private Storage 내 이미지 저장 경로
ocr_text	text	AI Pipeline의 OCR 결과
caption	text	BLIP 기반 Image Caption
category	text	사용자 이미지 폴더 분류 정보
search_text	text	임베딩 생성에 사용하는 검색용 텍스트
created_at	timestampz	이미지 등록 시각

events Table

events Table은 이미지에서 추출한 일정 정보를 저장하며 AI Pipeline이 Event Type을 none이 아닌 값으로 판단한 경우에만 Event Record를 생성한다. is_used는 쿠폰이나 예약과 같이 사용 여부를 관리해야 하는 Event에 활용하며 사용 완료 상태는 Frontend의 일정 표시와 알림에도 반영된다.

표 3.7: events Table Schema

Column	Type	설명
id	uuid	Event 고유 ID
image_id	uuid	Event가 추출된 이미지 ID
event_type	text	Event Type
event_date	date	Event 발생 날짜
event_time	time	Event 시작 시간
title	text	Event 제목
location	text	Event 장소
is_used	boolean	쿠폰 및 일정의 사용 완료 여부
created_at	timestampz	Event 생성 시각

image_embeddings Table

image_embeddings Table은 자연어 이미지 검색에 사용하는 이미지별 임베딩을 저장하며, image_id에 UNIQUE 제약을 적용하여 하나의 이미지에 하나의 검색용 임베딩이 존재하도록 구성하였다.

표 3.8: image_embeddings Table Schema

Column	Type	설명
id	uuid	임베딩 고유 ID
image_id	uuid	임베딩에 대응하는 이미지 ID
embedding	vector(1024)	KURE-v1 기반 Search Text Embedding

3.4.3 REST API 설계

Frontend와 Backend 간 통신은 REST API 기반으로 구성하였으며 이미지 등록 요청에는 multipart/form-data, 그 외 요청과 응답에는 JSON을 기본 형식으로 사용하였다.

표 3.9: 주요 REST API

Method	Endpoint	설명
POST	/images	이미지 등록 및 AI Pipeline 실행
GET	/images	사용자 소유 이미지 목록 조회
GET	/images/{image_id}	이미지 상세 조회
PATCH	/images/{image_id}/category	사용자 폴더 Category 수정
DELETE	/images/{image_id}	이미지 및 관련 데이터 삭제
GET	/images/{image_id}/events	특정 이미지의 Event 조회
GET	/events	사용자 전체 Event 조회
PATCH	/events/{event_id}/used	Event 사용 완료 여부 수정
GET	/search	임베딩 기반 자연어 이미지 검색

3.4.4 인증 및 이미지 접근 보안

사용자 인증에는 Supabase Auth를 사용하며 로그인 후 발급받은 Access Token을 모든 API 요청의 Authorization Bearer Header에 포함한다. Backend는 Supabase에서 제공하는 공개키를 이용하여 JWT 서명을 검증하고, Database 접근에 service_role 권한을 사용하므로 API 계층에서 Resource와 현재 요청자의 user_id를 함께 확인하여 자신의 데이터에만 접근할 수 있도록 하였다. 다른 사용자의 Resource에 접근하는 경우에는 Resource의 존재 여부가 노출되지 않도록 404로 처리하였다.

또한 이미지에는 쿠폰이나 예약 확인서 등 개인정보가 포함될 수 있으므로 Supabase Storage의 Private Bucket에 원본을 저장하고 Database에는 고정 URL 대신 storage_path만 저장하였다. 이미지 조회 시 소유권을 확인한 후 일정 시간 동안 유효한 Signed URL을 생성하며, jpg, jpeg, png, webp, heic 형식과 최대 15MB의 파일만 허용하였다.

3.4.5 AI Pipeline 연동 및 자연어 검색

이미지 등록 요청을 받으면 Backend는 원본을 Storage에 저장한 후 AI Pipeline에 분석을 요청하고, 생성된 OCR Text, Image Caption, Event, Search Text 및 Embedding을 각 Table

에 저장한다. 이미지 등록 시점에 분석 결과와 Embedding을 미리 생성함으로써 이후 조회와 검색에서 동일한 Pipeline을 반복 실행하지 않도록 하였다.

자연어 검색 시에는 AI Server의 `/embed-query` Endpoint를 통해 검색어를 KURE-v1 기반 1024차원 Query Embedding으로 변환하고, PostgreSQL의 `pgvector`와 `match_images` 함수를 이용하여 사용자 소유 이미지에 대해 코사인 유사도를 계산한다. 검색된 `image_id`를 이용하여 이미지 상세 정보와 Signed URL을 구성한 뒤 Frontend에 반환한다.

3.5 프론트엔드 시스템 구현

3.5.1 UI/UX 및 인증

Frontend는 React Native 기반 모바일 애플리케이션으로 구현하였으며, 사용자가 AI 분석 과정을 직접 의식하기보다 일반적인 갤러리처럼 이미지를 등록하고 검색하며 일정을 확인할 수 있도록 구성하였다. 사용자 인증에는 Supabase Auth를 사용하고 로그인 이후 발급된 Access Token을 공통 API 요청 함수에서 자동으로 Header에 포함하도록 구현하였다.

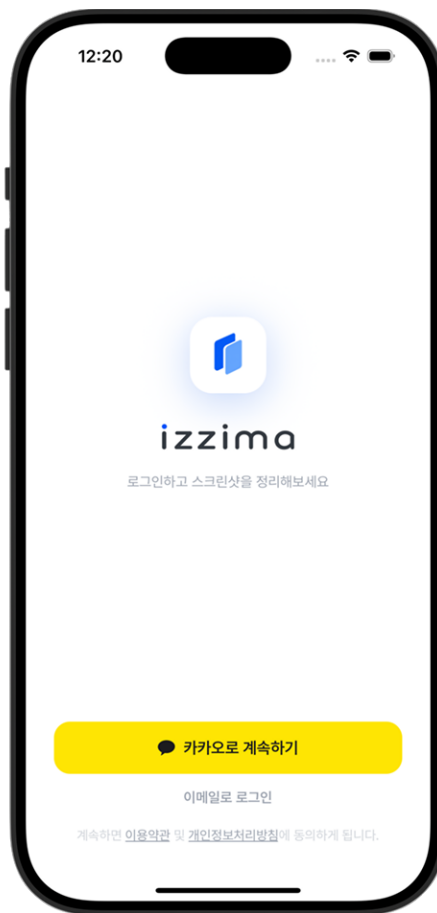


그림 3.4: 사용자 로그인 화면

3.5.2 이미지 업로드 및 중복 방지

사용자는 모바일 갤러리에서 여러 이미지를 선택하여 등록할 수 있으며, 업로드 화면에서는 선택된 이미지를 Grid 형태로 표시하고 각 항목을 대기, 업로드 중, 완료 상태로 구분하였다. 전체 업로드 진행률과 처리된 이미지 수도 함께 표시하여 여러 이미지를 등록하는 경우에도 현재 진행 상태를 확인할 수 있도록 하였다.

또한 동일한 이미지를 반복 등록하는 상황을 줄이기 위해 중복 방지 로직을 적용하였다. 갤러리 Asset의 고유 식별자인 `assetId`를 우선적인 비교 기준으로 사용하되, 사용자가 제한된 사진 접근 권한만 허용하여 `assetId`가 제공되지 않는 경우에는 파일 크기와 가로·세로 해상도의 조합을 대체 식별자로 사용하였다. 업로드에 성공한 이미지의 식별자는 `AsyncStorage`에 누적 저장하고, 이후 사용자가 갤러리에서 이미지를 다시 선택할 때 저장된 식별자 목록과 비교하여 이미 업로드한 이미지를 자동으로 제외하도록 구성하였다.

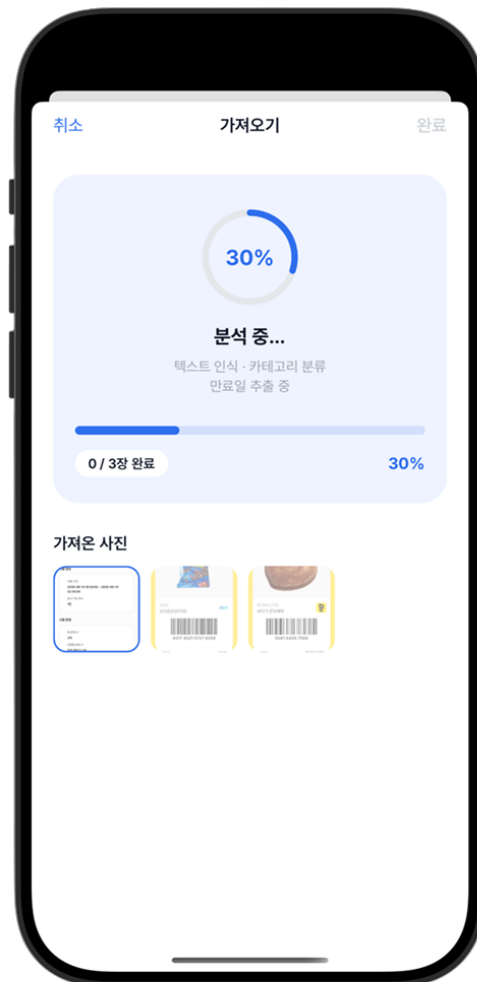


그림 3.5: 이미지 업로드 진행 화면

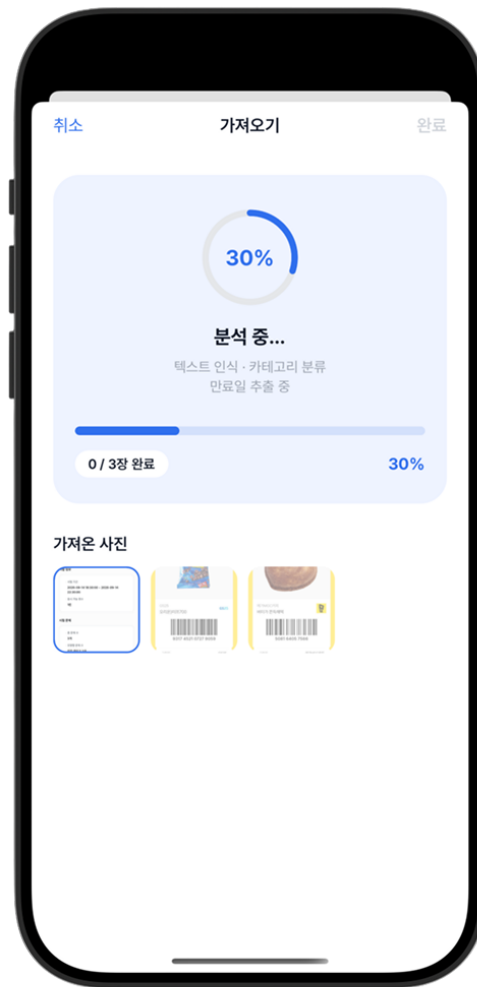
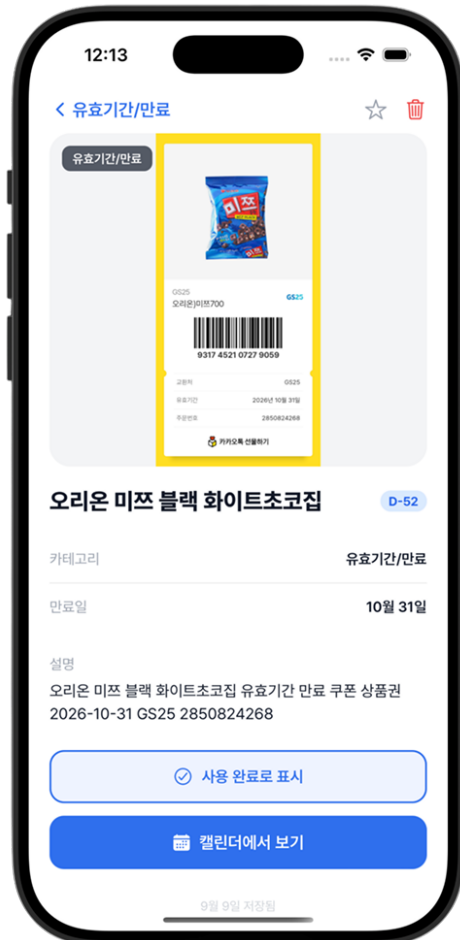


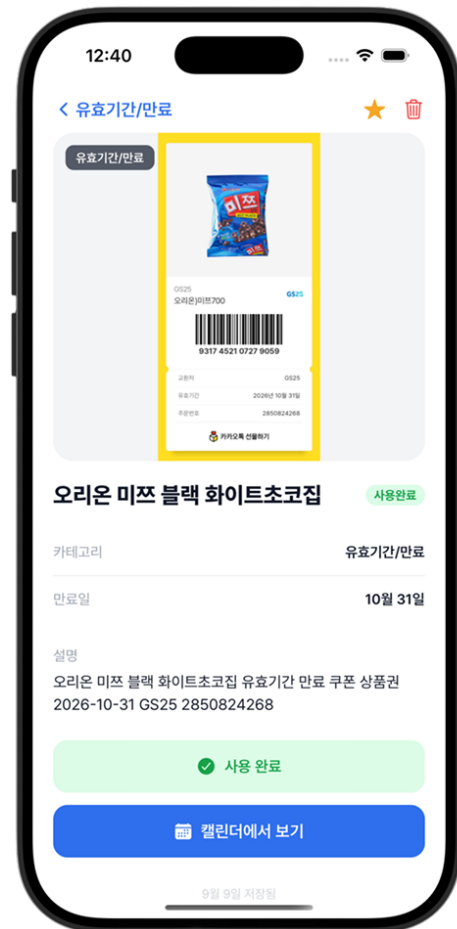
그림 3.6: 이미지 업로드 진행 화면

3.5.3 AI 분석 결과 및 이미지 상세 조회

업로드된 이미지는 Backend와 AI Pipeline을 거쳐 OCR, Image Caption 및 Event 정보가 생성된다. Frontend에서는 Event 존재 여부에 따라 상세 화면을 구분하여 일정 정보가 존재하는 이미지에는 Event 제목, 날짜, 장소 및 D-day를 표시하고, 일정 정보가 없는 일반 이미지에는 이미지 기본 정보를 간결하게 표시하였다. 또한 사용 완료 여부를 변경할 수 있도록 하였으며, Server 응답 전에 UI를 먼저 변경하는 Optimistic Update를 적용하고 API 호출 실패 시 기존 상태로 복구하도록 구성하였다. 사용 완료된 Event는 만료 임박 목록과 알림 대상에서도 제외된다.



(a) Event가 존재하는 이미지

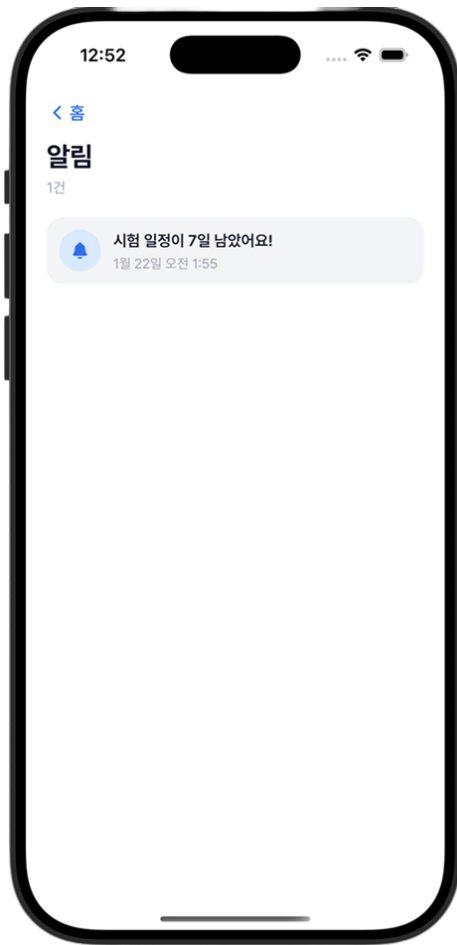


(b) 사용 완료 처리된 이미지

그림 3.7: 이미지 상세 화면

3.5.4 만료 임박 일정 알림

쿠폰과 예약처럼 시간에 민감한 Event를 사용자가 놓치지 않도록 expo-notifications를 이용한 로컬 알림 기능을 구현하였다. Event 날짜를 기준으로 일주일 전, 하루 전, 당일 오전 9시에 알림을 예약하며 사용 완료 상태로 변경된 Event는 이후 알림 대상에서 제외하였다.



(a) 로컬 알림



(b) 알림 배너

그림 3.8: 만료 임박 일정 알림

3.5.5 자연어 이미지 검색

사용자가 검색어를 입력하면 Frontend는 Backend의 GET `/search` API를 호출하며, Backend는 검색어를 KURE-v1으로 임베딩하고 기존 Image Embedding과의 코사인 유사도를 계산하여 관련성이 높은 이미지를 반환한다. Frontend에서는 검색 결과를 이미지 목록 형태로 제공하고 사용자가 원하는 이미지를 선택하여 상세 정보를 확인할 수 있도록 구성하였다.

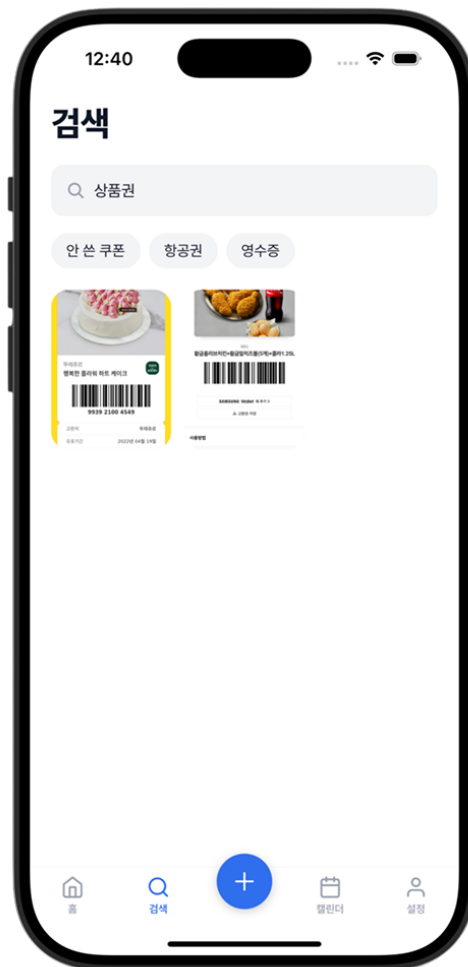


그림 3.9: 자연어 이미지 검색 화면

3.5.6 카테고리 폴더 관리 및 즐겨찾기

카테고리 폴더에서는 이미지를 길게 누르면 다중 선택 모드로 전환되며 선택된 이미지를 테두리와 Overlay 및 Check Icon으로 구분한다. 하단 Action Bar를 통해 전체 선택, 해제 및 삭제 기능을 제공하고 여러 삭제 요청은 `Promise.allSettled`로 병렬 처리하여 일부 요청이 실패하더라도 나머지 작업이 계속 수행되도록 하였다. 즐겨찾기는 별도의 Backend API 없이 `AsyncStorage`에 이미지 ID 목록을 저장하고 React Context를 통해 여러 화면에서 동일한 상태를 공유하며, 앱 실행 시 저장된 목록을 비동기적으로 불러와 기존 상태를 복원한다.



그림 3.10: 이미지 관리 기능

3.6 시스템 통합 및 서비스 구현

3.6.1 전체 데이터 흐름

사용자가 React Native 애플리케이션에서 이미지를 등록하면 Backend가 인증 및 파일 형식을 검증한 뒤 원본 이미지를 Private Storage에 저장하고 AI Pipeline에 분석을 요청한다. AI Pipeline은 OCR Text, Image Caption, Event, Search Text 및 Embedding을 생성하며 Backend는 이를 Database에 저장한다. 이후 사용자가 자연어 검색어를 입력하면 Query Embedding을 생성하고 pgvector를 이용하여 저장된 Image Embedding과 코사인 유사도를 계산한 뒤 관련성이 높은 이미지와 Signed URL을 Frontend에 반환한다.

3.6.2 일정 관리 흐름

AI Pipeline에서 일정 정보가 추출된 이미지는 `events` Table에 저장되며 Frontend는 이를 이용하여 D-day, 날짜 및 일정 정보를 표시한다. 사용자가 일정 또는 쿠폰을 완료한 경우 `is_used` 상태를 변경할 수 있으며 변경 결과는 Database와 Frontend에 함께 반영된다. 또한 사용 완료되지 않은 Event를 대상으로 날짜 기반 로컬 알림을 예약하여 만료 또는 일정 시점을 사용자에게 안내한다.

3.6.3 서버 및 배포 환경

AI Pipeline은 NVIDIA RTX A5000 GPU가 설치된 Ubuntu Server에서 구동하였으며 Qwen 계열 LLM의 추론에는 Ollama를 사용하였다. AI Server는 FastAPI 기반으로 구성되어 이미지 분석을 위한 `/analyze`, 검색 Query Embedding 생성을 위한 `/embed-query`, 서버 상태 확인을 위한 `/health` Endpoint를 제공하였다. Backend와 AI Server를 분리함으로써 Frontend가 AI 모델과 직접 통신하지 않도록 구성하였다.

제4장 연구 결과 분석 및 평가

4.1 평가 환경

4.1.1 평가 데이터 구성

Search Text 구성 방식에 따른 검색 성능을 비교하기 위해 먼저 12장의 이미지로 Validation Set을 구성하였다. 모바일 상품권 및 쿠폰, 영화 예매 내역, 항공권, 공연 예매 내역, 시험 및 학사 공지와 같은 Event 이미지와 일정 정보가 없는 일반 사진 및 스크린샷을 포함하였으며, Event Type은 expiration 2장, exam 1장, assignment_due 2장, reservation 1장, departure 2장, performance 2장, none 2장으로 구성하였다.

각 이미지에 대해서는 PaddleOCR, BLIP Image Captioning, Qwen 기반 Event 추출을 최초 한 번만 수행한 뒤 ocr_text, caption, event를 저장하였고, V1-V6 실험에서는 동일한 결과를 사용하여 Search Text 구성 방식만 변경하였다. 이후 Validation Set에 대한 과도한 최적화를 방지하기 위해 기존 실험에 사용하지 않은 10장의 이미지로 별도의 Test Set을 구성하였으며, Test 단계에서는 Validation에서 정의한 V1-V6를 수정하지 않고 그대로 적용하였다.

4.1.2 검색 질의 및 평가 지표

Validation Set과 Test Set 모두 각 이미지마다 실제 사용자가 해당 이미지를 다시 찾는 상황을 가정하여 3개의 자연어 검색 질의를 작성하였다. 따라서 Validation Set에서는 총 36개, Test Set에서는 총 30개의 질의를 사용하였으며 상품명, 날짜와 시간, 장소와 좌석 등의 세부 정보 및 일반 이미지의 시각적 의미 등 서로 다른 검색 단서가 포함되도록 구성하였다. 검색 성능은 Recall@1, Recall@3 및 MRR(Mean Reciprocal Rank)을 이용하여 평가하였다. 각 검색 질의마다 하나의 정답 이미지를 지정하였으므로 본 실험에서 Recall@K는 정답 이미지가 상위 K개의 검색 결과에 포함된 비율을 의미하며 Hit@K와 동일한 값을 갖는다. MRR은 각 질의에서 정답 이미지가 처음 등장하는 순위의 역수를 평균한 값이다.

$$\text{MRR} = \frac{1}{N} \sum_{i=1}^N \frac{1}{r_i} \quad (4.1)$$

여기서 N 은 전체 검색 질의 수이며, r_i 는 i 번째 질의의 정답 이미지 순위를 의미한다.

4.1.3 실험 환경

표 4.1: AI 및 검색 실험 환경

항목	설정
OCR	PaddleOCR
Image Captioning	BLIP
Primary LLM	Qwen3.5 9B
Fallback LLM	Qwen3.5 27B
Embedding	KURE-v1, 1024 dimensions
Similarity	Cosine Similarity
LLM Inference Server	Ollama
GPU	NVIDIA RTX A5000
Validation Set	12 images, 36 queries
Test Set	10 images, 30 queries

4.2 AI Pipeline 결과 분석

4.2.1 Event 정보 추출 결과

평가 이미지에 AI Pipeline을 적용한 결과 서로 다른 종류의 생활형 이미지에서 일정 및 세부 정보를 공통 Event Schema로 구조화할 수 있음을 확인하였다. 모바일 상품권에서는 상품명, 유효기간, 사용처 및 금액을 추출하였고 공연 및 영화 이미지에서는 제목, 관람 날짜, 시간과 장소를 구조화하였다. 항공권에서는 출발 날짜와 시간을 departure Event로, 시험 공지에서는 시험명, 날짜, 시작 및 종료 시간과 장소를 exam Event로 추출하였으며 예약번호, 좌석, 구매금액, 수령 방법 등 Event별 세부 정보는 metadata에 저장하였다.

4.2.2 Fallback 및 후처리 적용 결과

대부분의 평가 이미지는 Primary Model인 Qwen3.5 9B를 이용하여 처리하였으나 OCR Text에서 중요한 정보가 확인되지만 Primary Model 결과에서 해당 값이 누락되는 경우 Qwen3.5 27B를 추가 호출하였다.

예를 들어 공차 모바일 상품권에서는 OCR에 주문번호가 존재했지만 Primary Model 결과에서 해당 정보가 누락되어 Fallback 조건이 동작하였다. 또한 expiration Event에서는 OCR에 금액이 존재하지만 LLM이 amount를 누락하는 경우를 고려하여 “50,000원”, “3만

원”, “5천원”과 같은 표현을 정규식으로 추가 추출하였다. 현재 Event 각 필드에 대한 별도의 Ground Truth 기반 정량 평가는 수행하지 않았으므로 Event 추출 및 후처리 효과는 실제 사례를 중심으로 확인하였다.

4.3 Search Text 구성 방식 비교

4.3.1 Validation 결과

Validation Set에서 V1부터 V6까지 여섯 가지 Search Text 구성 방식을 비교한 결과는 다음과 같다.

표 4.2: Validation Set에서의 Search Text 구성 방식별 검색 성능

Version	Recall@1	Recall@3	MRR
V1	0.7222	0.9167	0.8282
V2	0.8056	0.9167	0.8685
V3	0.8333	0.9722	0.9037
V4	0.7778	0.9722	0.8750
V5	0.8889	1.0000	0.9398
V6	0.9167	1.0000	0.9537

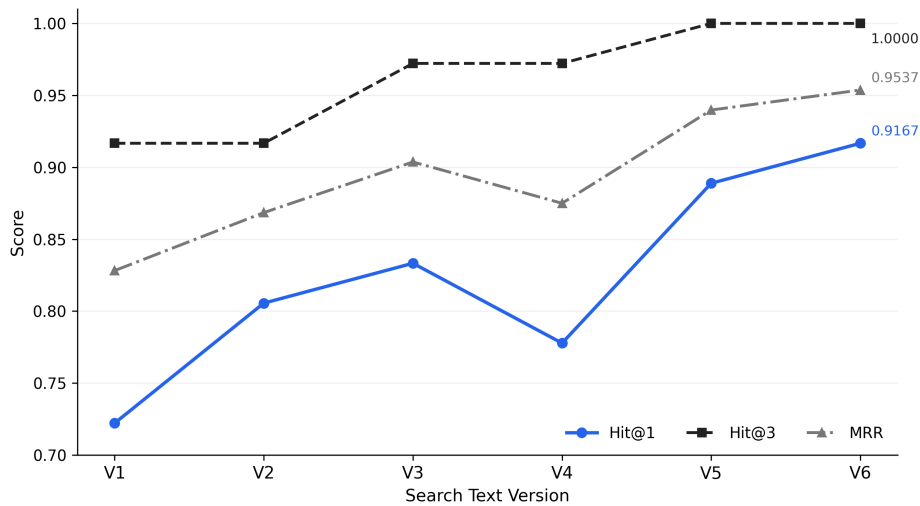


그림 4.1: Validation Set에서의 Search Text 구성 방식별 검색 성능 변화

V1에서 V2로 확장하면서 end_time과 metadata 정보를 추가한 결과 Recall@1과 MRR이 향상되었으며, V3에서는 Event Type을 사용자 검색 표현에 가까운 자연어로 확장하면서 추가적인 성능 향상을 확인하였다. 반면 모든 이미지에 BLIP Image Caption을 추가한 V4는 V3보다 Recall@1과 MRR이 감소하여 검색과 직접적인 관련성이 낮은 시각적 설명이 구조화된 핵심 정보의 의미를 희석할 수 있음을 확인하였다. 이후 none 이미지에 OCR과

Caption을 선택적으로 적용한 V5와 V6에서는 성능이 다시 향상되었으며, V6는 Recall@1 0.9167, Recall@3 1.0000, MRR 0.9537로 가장 높은 성능을 기록하였다. 이는 36개의 질의 중 33개에서 정답 이미지가 첫 번째로 검색되었고 모든 질의에서 정답 이미지가 상위 3개 이내에 포함되었음을 의미한다.

4.3.2 Test Set 최종 검증

Validation Set에서 Search Text 구성 방식을 비교한 이후 별도의 Test Set에 동일한 V1-V6를 적용하여 최종 성능을 검증하였다.

표 4.3: Test Set에서의 Search Text 구성 방식별 검색 성능

Version	Recall@1	Recall@3	MRR
V1	0.5000	0.7000	0.6466
V2	0.6000	0.8333	0.7333
V3	0.7000	0.9000	0.8153
V4	0.7333	0.8667	0.8162
V5	0.9333	1.0000	0.9667
V6	0.9333	1.0000	0.9667

Test Set에서도 V5와 V6가 Recall@1 0.9333, Recall@3 1.0000, MRR 0.9667로 가장 높은 성능을 기록하였으며 총 30개의 질의 중 28개에서 정답 이미지가 첫 번째로 검색되었다. V5와 V6가 동일한 결과를 보인 것은 Test Set의 none 이미지에서 유효한 OCR Text가 거의 존재하지 않아 두 방식 모두 주로 Image Caption을 사용했기 때문으로 분석하였다. 그러나 Validation Set에서는 V6가 V5보다 높은 성능을 보였고 Test Set에서도 성능 저하 없이 동일한 최고 성능을 유지하였으며, OCR Text가 존재하는 none 이미지에서 불필요한 Caption 노이즈를 제거할 수 있다는 점을 고려하여 최종 시스템에서는 V6를 선정하였다.

4.3.3 검색 오류 분석

최종 V6는 Test Set의 30개 질의 중 28개를 Top-1으로 검색하였으나 2개의 질의에서는 정답 이미지가 첫 번째로 검색되지 않았다. 대표적으로 동일 항공사의 출국 및 귀국 항공권처럼 항공사명, 출발지, 도착지와 관련 표현이 유사한 이미지는 Search Text 역시 임베딩 공간에서 가깝게 표현될 수 있다. 특히 “A에서 B로 이동”과 “B에서 A로 이동”처럼 방향성이 중요한 질의는 단일 임베딩이 필드 사이의 세부 관계까지 완전히 구분하기 어렵다는 한계가 있다. 향후에는 출발지, 도착지, 날짜 및 장소와 같은 구조화 Event 필드를 검색 조건으로 함께 이용하거나 임베딩 검색 이후 Reranking을 적용하여 개선할 수 있다.

4.4 시스템 구현 결과

최종 구현된 IZZIMA는 이미지 업로드부터 AI 분석, Event 정보 저장, 자연어 검색, 일정 관리 및 알림까지의 전체 흐름을 하나의 모바일 애플리케이션에서 수행할 수 있도록 통합하였다. 이미지 등록 시 AI Pipeline의 분석 결과가 Database에 저장되고 추출된 Event는 일정 관리와 알림에, Search Text와 Embedding은 자연어 검색에 활용된다. 이를 통해 Frontend, Backend, AI Pipeline 및 Database가 하나의 서비스 흐름으로 연동됨을 확인하였다.

4.5 개발 일정 및 역할 분담

본 프로젝트는 2026년 5월부터 9월까지 기획 및 시스템 설계를 시작으로 AI, Backend, Frontend 개발을 진행하였으며, 후반에는 시스템 통합, 성능 개선 및 최종 결과물 정리를 수행하였다.

개발구분	세부항목	5월				6월				7월				8월				9월			
		1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
기획	요구사항 분석 및 기능 정의																				
	시스템 구조 설계 및 기술 스택 선정																				
데이터 준비	Validation/Test 이미지 데이터셋 구축 및 평가 질의 구성																				
AI 개발	OCR 및 이미지 캡셔닝 모델 테스트																				
	Qwen 기반 Event 구조화 추출 및 후처리																				
	Search Text 구성 실험 및 KURE-v1 임베딩 적용																				
서버 개발	이미지 업로드 및 Storage API 구현																				
	AI Pipeline 연동 및 DB 저장																				
	pgvector 기반 자연어 검색 API 구현																				
	Event 조회 및 D-day 기능 구현																				
클라이언트 개발	이미지 업로드 및 자동 폴더 UI 구현																				
	자연어 검색 UI 및 검색 결과 카드 구현																				
	추천 카드 및 이미지 상세 화면 구현																				
통합 테스트	기능 통합 및 오류 수정																				
성능 개선	검색 정확도 개선 및 UI/UX 고도화																				
최종 마무리	최종 보고서 작성 및 발표 준비																				

그림 4.2: 프로젝트 개발 일정

프로젝트는 AI Pipeline, Backend 및 Database, Frontend를 중심으로 역할을 분담하였으며 개발 후반에는 모듈 연동과 통합 테스트를 함께 수행하였다.

학 번	이 름	구성원별 역할
202355621	문진서	AI Pipeline 설계 및 구현, PaddleOCR 기반 OCR 모듈 개발, Qwen Prompt Engineering 및 Event 추출 구조 설계, Search Text 생성 방식 설계 및 비교 실험, KURE-v1 임베딩 설계, Validation/Test 데이터셋 구축 및 자연어 검색 성능 평가
202355555	안지원	Supabase 데이터베이스 및 Storage 구축, CRUD 모듈과 이미지 업로드 API 구현, Backend 구조 설계 및 AI Pipeline 연동, 자연어 검색 API와 D-day 추천 기능 개발, 시스템 통합 및 테스트
202355652	이태경	React Native 기반 UI/UX 설계, React Navigation 및 화면 구조 구현, 홈 · 검색 · 캘린더 · 상세 화면 개발, Backend API 연동 및 사용자 인터페이스 개선, 기능 통합 테스트

그림 4.3: 구성원별 역할 분담

4.6 오류 사례 및 한계 분석

OCR은 LLM이 사용하는 주요 입력이므로 작은 글자, 저화질 이미지, 복잡한 배경 또는 숫자와 영문이 혼합된 예약번호에서 문자열이 잘못 인식되거나 누락되면 최종 Event 추출에도 영향을 줄 수 있다. 또한 모바일 상품권이나 티켓에는 구매일, 유효기간, 취소 기한 등 여러 날짜가 동시에 존재할 수 있어 문맥이 불분명한 경우 LLM이 잘못된 값을 선택할 가능성이 있다. OCR에 정보가 존재하더라도 LLM이 일부 metadata를 누락할 수 있어 Fallback Model과 규칙 기반 후처리를 적용하였지만 모든 필드를 규칙적으로 검증하기는 어렵다.

현재 시스템은 10개의 Event Type을 기반으로 하므로 새로운 종류의 일정이나 하나의 이미지에 복수의 독립적인 Event가 포함된 경우 현재 Schema만으로 모든 정보를 표현하기 어렵다. 또한 PaddleOCR, BLIP, Qwen 및 KURE를 순차적으로 사용하기 때문에 단순 OCR 시스템보다 처리 시간이 증가하며 Fallback Model이 호출될 경우 추가적인 LLM 추론 시간이 발생한다.

검색 역시 하나의 Search Text를 단일 임베딩으로 표현하므로 출발지와 도착지처럼 필드 사이의 관계가 중요한 경우 세부적인 차이를 구분하는 데 한계가 있다. 아울러 현재 평가는 Validation 12장과 Test 10장의 제한된 데이터로 수행하였으며 일부 Event Type은 평가 데이터에 포함되지 않았으므로 향후 더 다양한 실제 이미지와 질의를 이용한 추가 검증이

필요하다.

Frontend는 Backend API를 통해 이미지와 일정 데이터를 조회하므로 Backend의 응답 지연이 사용자의 체감 대기 시간으로 직접 이어지는 한계가 있다. 이미지 목록 조회 시에는 Client와 Backend 간 통신뿐만 아니라 Supabase PostgreSQL 조회와 Storage의 Signed URL 발급 과정이 추가로 필요하다. 실제 측정 결과 Backend Instance가 활성 상태일 때 단순 요청은 약 0.33초가 소요되었으나, Database 조회와 다건의 Signed URL 발급이 포함된 요청에서는 체감 지연이 약 2초까지 증가하였다. 또한 Render 무료 Plan에서는 일정 시간 요청이 없을 경우 Instance가 유휴 상태로 전환되어 이후 최초 요청에서 수십 초의 지연이 발생하는 경우가 확인되었다.

이러한 반복적인 네트워크 요청으로 인한 지연을 줄이기 위해 React Query 기반 Cache를 적용하여 화면 간 조회 결과를 공유하고, 동일한 데이터를 다시 조회할 때 이전 응답을 우선적으로 표시하도록 개선하였다. 그러나 이러한 방식은 반복 조회에서 발생하는 지연을 완화할 수 있을 뿐, 세션 내 최초 조회나 Server의 유휴 상태 해제 과정에서 발생하는 지연까지 제거하지는 못한다. 향후에는 Backend Hosting 환경과 데이터 조회 및 Signed URL 발급 과정을 개선하여 최초 화면 진입 시 발생하는 응답 지연을 줄일 필요가 있다.

이미지 중복 방지 기능은 Client에 저장된 이미지 식별자를 기준으로 동작한다는 한계가 있다. 기본적으로 갤러리 Asset의 고유 식별자인 `assetId`를 사용하지만, 제한된 사진 접근 권한으로 인해 해당 값이 제공되지 않는 경우에는 파일 크기와 가로, 세로 해상도의 조합을 대체 식별자로 사용한다. 따라서 서로 다른 이미지가 우연히 동일한 파일 크기와 해상도를 갖는 경우 동일한 이미지로 잘못 판단될 가능성이 있다. 또한 업로드 이력을 Client의 AsyncStorage에 저장하므로 애플리케이션을 재설치하거나 다른 기기에서 동일한 이미지를 업로드하는 경우까지 방지할 수는 없다. 향후에는 이미지 파일의 실제 Byte를 기반으로 Hash 값을 생성하거나 Server 측에 이미지 콘텐츠 기반 중복 검사 기능을 추가하여 이러한 한계를 개선할 수 있다.

제5장 결론 및 향후 연구 방향

5.1 연구 결과 요약

본 프로젝트에서는 스마트폰 갤러리 이미지에 포함된 생활형 정보를 자동으로 분석하고 이를 자연어 검색과 일정 관리에 활용할 수 있는 AI 기반 개인 갤러리 관리 서비스 IZZIMA를 구현하였다. AI Pipeline은 PaddleOCR과 BLIP을 이용하여 텍스트 및 시각적 정보를 추출하고 Qwen 계열 LLM을 통해 Event Type과 구조화된 Event 정보를 생성하며, Primary-Fallback 추론 구조와 규칙 기반 후처리를 적용하여 일부 정보 누락을 보완하였다. 자연어 검색에는 구조화 Event를 기반으로 생성한 Search Text와 KURE-v1 임베딩을 사용하였으며, Validation Set에서 가장 높은 성능을 보인 V6를 별도의 Test Set에 적용한 결과 Recall@1 0.9333, Recall@3 1.0000, MRR 0.9667을 기록하였다. 또한 React Native Frontend, FastAPI Backend, Supabase Database 및 Storage와 AI Pipeline을 통합하여 이미지 등록, Event 추출, 자연어 검색, 일정 관리 및 알림까지 전체 서비스 흐름을 구현하였다.

5.2 향후 연구 방향

향후에는 평가 이미지와 자연어 질의를 확대하여 다양한 실제 갤러리 환경에서 AI Pipeline과 검색 성능을 추가적으로 검증할 필요가 있다. 또한 Event Type과 metadata Schema를 확장하고 하나의 이미지에서 복수의 Event를 추출할 수 있도록 구조를 개선할 수 있다. 검색 측면에서는 임베딩 기반 1차 검색 이후 날짜, 장소, 출발지와 도착지 등의 구조화 Event 필드를 Filtering 또는 Reranking에 활용하여 유사 이미지 사이의 세부적인 구분 능력을 향상시킬 수 있으며, 멀티모달 임베딩을 적용하여 이미지의 시각 정보 자체를 검색에 직접 활용하는 방법도 고려할 수 있다.

서비스 측면에서는 Backend Hosting 환경과 데이터 조회 및 Signed URL 발급 과정을 최적화하여 최초 화면 진입 시 발생하는 응답 지연을 줄일 필요가 있다. 또한 현재 Client의 업로드 이력을 기반으로 동작하는 이미지 중복 방지 방식을 개선하여, 이미지 파일의 실제 Byte를 기반으로 Hash 값을 생성하거나 Server 측에 이미지 콘텐츠 기반 중복 검사 기능을 추

가함으로써 애플리케이션 재설치나 다른 기기에서의 재업로드까지 처리할 수 있도록 확장할 수 있다.

장기적으로는 사용자의 검색 기록과 이미지 이용 이력을 활용한 개인화 기능과 모델 경량화, 조건부 Pipeline 및 결과 Caching 등을 적용하여 실제 서비스 환경에서의 검색 편의성과 처리 효율을 개선할 수 있다.

제6장 참고 문헌

1. Du, Y., Li, C., Guo, R., et al., “PP-OCR: A Practical Ultra Lightweight OCR System,” arXiv preprint arXiv:2009.09941, 2020.
2. Li, J., Li, D., Xiong, C., Hoi, S., “BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation,” Proceedings of ICML, 2022.
3. Bai, J., Bai, S., Yang, S., et al., “Qwen Technical Report,” arXiv preprint arXiv:2309.16609, 2023.
4. Reimers, N., Gurevych, I., “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” Proceedings of EMNLP, 2019.
5. PostgreSQL Global Development Group, “pgvector Documentation.”
6. FastAPI Documentation.
7. Supabase Documentation.
8. PaddleOCR Documentation.
9. BLIP Documentation.
10. Qwen Documentation.
11. KURE Model Documentation.